

25th June GFG POTD

Increasing Digits Numbers (Recursion + Backtracking Notes)

Problem Statement

Given an integer n , return all n -digit numbers whose digits are in **strictly increasing order** from left to right.

Example

```
Input: n = 1
Output: [0,1,2,3,4,5,6,7,8,9]

Input: n = 2
Output:
[12,13,14,15,16,17,18,19,
 23,24,25,26,27,28,29,
 ...
 89]
```

Step 1: Brute Force Idea

Generate all n -digit numbers.

For each number:

1. Extract digits.
2. Check if every digit is smaller than the next digit.
3. If yes, add it to the answer.

Example:

```
12 -> valid
13 -> valid
21 -> invalid
44 -> invalid
135 -> valid
```

Complexity

For $n = 3$

100 → 999

We check hundreds of numbers.

In general:

$O(10^n \times n)$

Very inefficient.

Step 2: Better Observation

Instead of generating all numbers and rejecting most of them:

Generate only valid numbers.

Example:

For first digit:

1

Next digit cannot be:

0 or 1

It must be:

2..9

If current number is:

13

Next digit must be:

```
4..9
```

This naturally forms a recursive tree.

Logic Building

Think of the problem as:

```
Choose digits one by one.
```

Rules:

```
After choosing digit d,  
next digit must be chosen from d+1 to 9.
```

Example:

```
1  
├── 2  
│   ├── 3 -> 123  
│   ├── 4 -> 124  
│   └── ...  
├── 3  
│   ├── 4 -> 134  
│   └── ...  
└── ...
```

This is exactly a recursion tree.

Why Recursion?

Whenever a problem says:

```
Choose something  
Then choose the next thing  
Then choose the next thing  
...
```

recursion is usually a good fit.

General structure:

```
def dfs(state):  
    if solution formed:  
        save answer  
        return  
  
    for choice in possible_choices:  
        make_choice()  
        dfs(new_state)  
        undo_choice()
```

What is Backtracking?

Backtracking means:

```
Choose  
Explore  
Undo
```

Pattern:

```
path.append(choice)    # choose  
  
dfs(...)  
  
path.pop()            # undo
```

Example:

```
path = [1]  
  
append(2)  
path = [1, 2]  
  
explore all possibilities  
  
pop()  
path = [1]
```

Now we can try another choice.

When to Use Backtracking?

Look for words like:

```
Generate all  
Find all  
Return all  
Combinations  
Permutations  
Subsets  
Possible arrangements  
Possible paths
```

Examples:

Subsets

```
[1,2,3]  
  
[]  
[1]  
[2]  
[3]  
[1,2]  
...
```

Permutations

```
123  
132  
213  
231  
312  
321
```

N-Queens

```
Place queens in every valid way.
```

Sudoku

Try a number
If invalid, undo and try another

Generic Backtracking Template

```
def backtrack(path):  
  
    if goal_reached:  
        ans.append(path)  
        return  
  
    for choice in choices:  
  
        path.append(choice)  
  
        backtrack(path)  
  
        path.pop()
```

Memorize this.

Most backtracking problems follow this exact structure.

Applying It Here

State:

Current number formed
Last digit chosen
Length formed so far

Choice:

Choose next digit from
(last_digit + 1) to 9

Base Case:

```
Length == n
```

Add number to answer.

Efficient Solution

```
class Solution:

    def backtrack(self, n, last_digit, num, length, ans):

        if length == n:
            ans.append(num)
            return

        for digit in range(last_digit + 1, 10):

            self.backtrack(
                n,
                digit,
                num * 10 + digit,
                length + 1,
                ans
            )

    def increasingNumbers(self, n):

        if n == 1:
            return list(range(10))

        ans = []

        for start in range(1, 10):
            self.backtrack(
                n,
                start,
                start,
                1,
                ans
            )

        return ans
```

Understanding the Initial Call

```
for start in range(1, 10):  
    backtrack(n, start, start, 1, ans)
```

Meaning:

Try every possible first digit.

Example:

start = 1

Current number:

1

Generate:

12
13
14
...
19

Then:

start = 2

Generate:

23
24
25
...
29

and so on.

Recursion Mental Model

Always ask:

1. What is my current state?

Example:

```
Current number = 135
Length = 3
Last digit = 5
```

2. What choices do I have?

```
6,7,8,9
```

3. What is the base case?

```
Length == n
```

4. What happens after choosing?

```
Add digit
Move deeper
```

Real-World Patterns to Identify Recursion

Pattern 1: Decision Tree

Keywords:

```
Choose
Pick
Select
Include/Exclude
```

Problems:

Subsets
Combinations
Target Sum
Partition

Pattern 2: Generate All Possibilities

Keywords:

Generate all
Return all
List all

Problems:

Permutations
Subsets
N-Queens
Parentheses

Pattern 3: Tree Traversal

Keywords:

Left subtree
Right subtree
Children

Problems:

DFS
Binary Tree
Graphs

Pattern 4: Break Into Smaller Same Problem

Keywords:

Solve smaller version first

Examples:

```
factorial(n)
factorial(n-1)

fib(n)
fib(n-1) + fib(n-2)
```

Recursion Checklist

Whenever stuck:

1. What is the state?
2. What are my choices?
3. What is the base case?
4. After choosing, what changes?
5. Do I need to undo the choice?

If answer to #5 is YES:

Use Backtracking

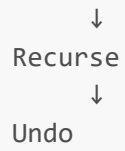
If answer to #5 is NO:

Simple Recursion is enough

Golden Rule

Most backtracking questions can be solved by thinking:

```
Current State
  ↓
Possible Choices
  ↓
Choose
```



```
graph TD; A[ ] --> B[Recurse]; B --> C[Undo];
```

That's the entire idea behind:

- Subsets
- Combinations
- Permutations
- N Queens
- Sudoku
- Word Search
- Rat in a Maze
- Increasing Digits Numbers